Platform

Trunk section: §6.10 Platform, with §9 Deployment, backup and restore.

Talks to: every domain. It gives them the front door and login, the one scheduler, configuration, health and alerts, backup and restore, deployment and the page shell.

Built on: today's architecture (the programs, the shared state, the environment variables), MONITOR (every job and its schedule), SETTINGS (every setting, live and dead), DATACHECK (judging sources by content), PAGES (the suite and landing pages), the code repo's `docs/BACKUPS.md`, `docs/USER-GUIDE.md`, `src/auth/index.js`, `quant-platform/chart/auth.py`, `deploy-tools.sh`, `deploy/tool-ctl.sh`, `scripts/backup-r2.js`, `scripts/restore-r2.js`; the helper scripts of the screener tools and qp, and the journal's OPERATIONS (§7.1 says where each goes).

Contents

1. Front door and login
2. The scheduler
3. Configuration
4. Health and alerts
5. Backup
6. Restore
7. Deployment and supervision
8. The page shell
9. Contracts
10. How the rebuild is proven

---

1. Front door and login

Legacy

Every program listens on a **public port of its own**, over plain HTTP (thirteen: the five running tools, the archive answering on the six stopped tools' ports, qp 8765, the journal 3100; the per-tool scorer ports are unused, the scorers being off). Until the 2026-10 review none asked who was calling (**S-1**): anyone who found a port could change settings, keys and strategies, and qp's log already showed internet scanners on 8765.

The fix that runs today, `src/auth/index.js` (Node) and `chart/auth.py` (qp, pure ASGI so its WebSocket is covered):

Part	Rule
`data/auth.json`	gitignored, mode 0600, written by `scripts/set-password.sh`: `{ v, salt, hash, secret }`, the password as a **scrypt** hash (N 16384, r 8, p 1) and a random signing secret; re-read when it changes, so a new password applies everywhere at once
`td_auth` cookie	`<expiry>.<HMAC-SHA256(secret, "td1.<expiry>")>`, **30 days**; a cookie, not Basic auth, because cookies ignore the port: signed in on one program, signed in on all. Flags `HttpOnly; SameSite=Lax`; not `Secure`, because nothing is served over HTTPS
let through	`/login`, `/logout`; calls from the box itself (127.0.0.1, ::1) that did not pass a proxy (the programs call each other; the backup; the deploy's health probes)
no password set	**locked, not open**: every outside request gets a page saying how to set one
a new password	writes a new secret, signing everyone out
minimum length	10 characters

New design

- **One HTTPS address** (Cloudflare Tunnel or Caddy with a certificate) in front of desk-web and quant; **every other port closed** to the internet.
- **The same login** (scrypt hash, signed cookie, locked when unset), served by desk-web and checked by quant with the same secret, with a session list and "sign out everywhere".
- The cookie is `Secure; HttpOnly; SameSite=Lax`, and a request that changes anything must carry the desk's own `Origin` (no cross-site form or script can act as the trader).
- Services call each other on the private network or localhost only.
- Login attempts are rate-limited and logged.

---

2. The scheduler

Legacy

Jobs are spread over **five tool processes** (each with its own `node-cron` schedule, stored in its database), **pm2 cron** (the R2 backup at 23:20 UTC), and a **hand-installed systemd timer** (qp's nightly chain at 22:30 UTC). Each tool's Monitor tab shows its jobs (toggle, edit the cron, reset), but **run history is in memory only** and lost on restart, and nothing alerts when a job fails or does not run.

Every legacy job (New York time, `1-5` = weekdays):

Job	Schedule	Where today
Discovery: pre-market	`*/30 4-8 * * 1-5`	each tool
Discovery: open	`*/5 9 * * 1-5`	each tool
Discovery: session	`*/15 10-15 * * 1-5`	each tool
Discovery: close	`0 16 * * 1-5`	each tool
Refresh live cards	`*/5 4-16 * * 1-5`	each tool
Shortlist auto-rule	one minute before R1	each tool
R1 capture	the tool’s capture time	each tool
R2 snapshots	`26,31,36,41,46,51,56 9 * * 1-5` and `1 10 * * 1-5`	each tool
R3 end-of-day capture (and R4)	`5 16 * * 1-5`	each tool
Scorer auto-train	`20 16 * * 1-5`	each tool
Apply tool schedule changes	`50 16 * * 1-5`	T1 only
Midnight r0 flush	`0 0 * * *`	each tool
O’Neil nightly chain	22:30 UTC, Mon–Fri	systemd timer
R2 backup	23:20 UTC, Mon–Fri, and at every deploy	pm2 cron
Journal account fetch	each minute, per account interval	the journal
qp signal watcher	every 60 s, 09:25–16:05	qp, when started

Rules kept: a paused tool skips scans (captures still run); no overlapping scans; errors logged with the stack. Faults: a job’s id was its slugified name, so a capture-time change gave the R1 job a new id and dropped its custom schedule.

New design

desk-worker is the only scheduler. Every job is a row:

Column	Meaning
`job`	a stable id (never derived from a name or time)
`domain`	which domain owns it
`schedule`	cron in New York time, or “per tool” (capture minutes from the tool rows)
`calendar`	trading days only, or every day
`enabled`	on / off
`expected_within`	how late a run may be before it counts as missed
`max_duration`	how long before a run counts as hung

Every run is a row: start, end, result, counts, error with stack. **A failed, missed or hung run sends a push alert**; the Monitor page shows every job of every domain, its last runs and its next run. The research worker, backup and quant’s alert cycles are jobs like any other: no pm2 cron, no systemd timer, no loop of its own in any service. The trading calendar (holidays, half days) decides which days run.

- **No overlap.** A job never runs twice at once: a run still going when its next slot comes skips that slot, recorded as `skipped` (the legacy scan guard, made general).
- **Dependencies, not clock guesses.** A job can wait for another: training waits for R3, the backup for the night’s research run. Today the backup runs at a fixed 23:20 UTC, fifty minutes after the nightly chain starts, whether or not the chain has finished.
- **Missed is said.** A job whose slot passed while desk-worker was down is recorded as `missed` and alerts; a photo is never taken late as if on time (REGISTERS.md §4.2).

The jobs of the new desk (New York time, trading days):

Job	Schedule	Domain
discovery	04:00–09:00 every 30 min; 09:00–10:00 every 5; 10:00–16:00 every 15; 16:00	tools and screeners
re-quote (and re-score)	every 5 min, 04:00–16:00	card
research: new tickers	every 5 min, 04:00–16:00, while any are waiting	research data
research: provisional M	every 30 min, 09:30–16:00	research data
quant alert cycle	every minute, 09:25–16:05, while alerts are on	quant
shortlist auto-rule	one minute before each tool’s photo	registers
R1 photo	each tool’s capture minute	registers
R3 outcomes	16:05 (half days: 15 minutes after the close)	registers
research: training and scorecards	after R3 (about 16:20)	learning
research: the night (O’Neil chain, SIC)	18:30	research data
backup and verification	after the night’s research run	platform
journal sync	each account’s own interval	journal
health checks	each adapter’s cadence; all of them before 04:00	platform
restore rehearsal	monthly	platform

R2 and R4 are views: no job writes them.

---

3. Configuration

Legacy

Settings live in each tool’s database (`settings` table), validated by `POST /api/settings` (`SETTING_RULES`). Live settings:

Group	Setting	Default	Range
Hot sector	`hotImmediateThreshold`	60	0–100
	`hotSustainedThreshold`	40	0–100
	`hotSustainedSessions`	3	1–20
	`hotFloorThreshold`	20	0–100
	`coolOffDays`	2	0–30 (counts sessions)
Sector bias	`sectorBullishThreshold`, `sectorBearishThreshold`	20, −20	0–100, −100–0
Tradability floor	`minPrice`, `minAvgVolume`, `minAtr`, `minAtrPct`	1, 1,000,000, 1, 3	(0 switches a leg off)
Shortlist auto-rule	`shortlistMinScore`, `shortlistTopN`	70, 5	0–100, 1–50
Scorer	`scorerEntryTime`, `regimeSampleThreshold`	9:40, 150	—
Keys	`finnhubApiKey`, `alpacaApiKey`, `alpacaApiSecret`, `alpacaAccountUrl`, `alpacaMarketFeed`	—	(the page has no inputs: keys go in `data/keys.json`)

31 dead keys accepted but never read by the tools (`trading_*` and aliases, `grading_*`, `analysis*`, `aiApiKey` / `aiModel`: D-T1). Values read as `value || default`, so a saved 0 acts as the default (D-T2). Some help texts are wrong (D-T3, D-T5). Each tool keeps its own floor, though the page calls it “every screener, every tool”.

Shared files: `tools.config.json` (repo) + `data/tools.local.json` (Manage tools), `data/keys.json`, `data/auth.json`, `data/r2.json`, `quant-platform/.env` and `.default-feed`, environment variables for every path and URL.

New design

Kind	Where	Checked
Settings (thresholds, floor, auto-rule, default feed)	one `settings` table: each key with scope (desk, or per tool), type, range, default, description, **version**	on save, with the reason; 0 is a value, not “use the default”
Tools and screeners	their tables (TOOLS-AND-SCREENERS.md)	on save
Secrets	one secret store (a 0600 file or a managed store)	set and tested from Settings; never shown again
Deployment	environment variables (database URL, ports, paths)	at start; a missing one stops the service with its name

Dead settings are not carried over. Every settings page is generated from the same definitions the validator uses, so help text cannot disagree with the code. A threshold change creates a new version, and results record which version produced them.

---

4. Health and alerts

Legacy

The data check: one button, up to two minutes, asking every source for data that cannot be empty and judging the content (DATACHECK). qp’s `feedcheck` at deploy. qp’s `run_status.py` and `run_fields.py` from a shell. No alerts.

New design

Signal	Source	Alert
source health	scheduled checks (SOURCES.md §6)	red
job runs	the scheduler	failed, missed or hung
fact freshness	each fact family's clock, counted in trading sessions (e.g. M and groups not from the last completed session; earnings not refreshed in 8 days), so a weekend or a holiday never reads as stale	amber, then red
service health	each service's `/health` (database reachable, last scan, queue depth)	down
disk, memory	the box	under thresholds
backup	the nightly verification	not clean

One **health page** shows every signal with its fix. **Push alerts** go to the phone (ntfy, Pushover or Telegram: the trader's choice), each with what failed, since when, and the fix. An alert is not repeated while its cause stands, and a recovery is announced.

---

5. Backup

Legacy

`scripts/backup-r2.js`, under pm2 as `r2-backup`, **weekdays 23:20 UTC** and at every deploy, to a Cloudflare R2 bucket (`trade-desk-backup`; credentials in `data/r2.json`, 0600). About 17 MB compressed a night.

```
trade-desk-backup/
├── daily/<YYYY-MM-DD>/            manifest.json (files, sizes, sha256, failures)
│   ├── data/tradedesk.db.gz, data/t2.db.gz … t11.db.gz   every tool, running or archived
│   ├── data/<shared>.json.gz       industry map, O'Neil files, shortlist, registry …
│   ├── data/journal/journal.db.gz
│   └── qp/platform.db.gz
├── monthly/<YYYY-MM>/             the first COMPLETE night of each month; manifest written
last
└── analysis/<tool>/<register>/<day>.csv.gz   R1 R2 R3A R3B R4A R4B Shortlist
```

Databases copied with SQLite's **online backup API** (consistent while written). **Never backed up:** `keys.json`, `r2.json`, `journal-accounts.json`, `auth.json` (secrets). **Not backed up, rebuildable:** the bar caches, the trained models (`src/scoring/outputs*`), qp's operations log. Retention by an R2 lifecycle rule on `daily/` (60 days recommended); `monthly/` and `analysis/` kept. Checked by `pm2 logs`, the deploy's line, and the manifest's `failures`. It replaced a GitHub backup that pushed API keys in plain text (**S-2**).

New design

Item	Rule
what	a database dump (Postgres `pg_dump`, or each SQLite file through the online backup API), the Parquet analysis exports, strategy JSON exports, **trained models** (rows in the database now)
when	nightly, as soon as the night's research run ends (a dependency, not a clock time); and before every migration
layout	the legacy `daily/`, `monthly/`, `analysis/` layout and retention, kept
verified	the dump is restored into a scratch database and its row counts compared with the live ones; the manifest records the result; a mismatch is a red alert
secrets	never in the backup; the trader keeps their own copy of the secret store

---

6. Restore

Legacy

Never writes over live data: `node scripts/restore-r2.js <date> [file|all]` downloads into `restore/<date>/`; putting a file back is manual, process by process (`pm2 stop tool-T2`, copy, delete the `-wal`/`-shm` files, `pm2 start`), with the right owner for each file (`tool-Tn`, `archive`, `journal`, `qp`). A new box: clone, deploy (it starts empty), set the R2 keys, restore everything, stop all, copy files into place, start all, set the password, re-enter the keys and the journal's accounts. The download step is tested (`tests/backup.r2.test.js`, against a stand-in bucket); a full restore onto a box is not on record and not scheduled. Models must be retrained by hand from the registers.

New design

Command	Does
`desk restore <date>`	stops the services; restores the database from that night's dump; runs migrations; starts the services; runs every health check; prints row counts against the manifest
`desk restore <date> --to scratch`	restores into a scratch database for inspection, touching nothing live
`desk restore --list`	the nights and months available, with each manifest's result

A monthly rehearsal (a scheduled job) restores last night's backup into the scratch database and reports the row counts; a failure is a red alert. **A new box** is: install, set the secret store, `desk restore <date>`. Nothing else is copied by hand.

---

7. Deployment and supervision

Legacy

`./deploy-tools.sh`: pull, stop, start and health-check everything under pm2; `deploy/tool-ctl.sh` is the one place a tool is started (clean environment, memory caps, `pm2 save`). Lessons recorded in the script:

- **It pulls itself.** Bash keeps reading the old file, so a change to the script took effect one deploy late; it now re-runs itself when it changed (guarded against looping).
- **The registry is read after the pull**, or a deploy starts the previous deploy's tools.
- **A guard before** `git reset --hard`: deploying from a branch with an older `quant-platform/` once silently reverted it.
- **One supervisor**: two supervisors once caused a 113,854-restart incident; nothing may run under systemd as well as pm2 (the nightly timer is the one exception, Q-N5).
- A qp restart took 3 min 20 s (a start-up check refused the port while old connections lingered); fixed.
- Memory: on a ~912 MB box, nothing starts with less than 180 MB free.

No CI: tests run when someone runs them.

New design

Step	Rule
CI	every pull request runs every test (Node, Python, the library's `run_all`, fixtures for every adapter and file format); red blocks the merge
Release	a merge to main builds a versioned release
Deploy	one command on the box: fetch the release, back up, run migrations, restart services in order (database, desk-worker, desk-web, quant, front door), check each `/health`; stop at the first failure and say why; roll back to the previous release on request
Supervisor	one (systemd units, or pm2), never both; memory limits per service
Logs	structured, per service, rotated; the last errors shown on the health page
The photo window	a deploy within 15 minutes of any tool's capture minute on a trading day is refused unless forced

7.1 The legacy scripts, and where each goes

Legacy	New home
`scripts/set-password.sh`, `set-r2-keys.sh`, `set-alpaca-keys.js`, `sync-qp-env.js`	`desk secret set <name>`: one secret store, read by every service, so nothing is copied between files
`deploy-tools.sh`, `deploy/tool-ctl.sh`, `deploy/qp-restart.sh`, `deploy/add-swap.sh`	`desk deploy` (§7); tools are rows, so no tool is started on its own; swap is set once at install
`show-screeners`, `check-screeners`, `why-empty`, `try-filter`, `field-values`, `verify-tv-fields`, the `probe-*` scripts	the builder's Test (`count`, `totalCount`, which rule empties it), the adapter's live column list, and the scheduled health checks
`audit-floor`, `purge-sub-floor`, `backfill_signals`, `rebuild_training`, `convert_legacy_to_r4.py`, `split-tool-history`	one-time repairs become versioned migrations and the import of old data (MIGRATION.md §3); R4 is a view, so nothing needs rebuilding
`analyse-bias`, `analyse-news-chain`, `card_lookup.py`, `verify_irdm_score.py`, `explain_score.py`	the Parquet exports and the API (a card as of a day, the R1 register, the score's breakdown)
qp `run_status.py`, `run_fields.py`	the research checks on the health page; `desk fields <ticker>`: every CANSLIM field with its status and reason
qp `run_daily.py`, `run_backfill.py`, `run_edgar.py`, `run_13f.py`, `run_sic.py`	the research worker's runs and steps, started by the scheduler or `desk research <step>`
qp `tools/compare_server.py`, `gen_catalog.py`, `check_strategies.py`, `explain_strategy.py`, `seed_from_db.py`, `recover_strategies.py`, `rank_sweep.py`; `exec-compare.js`, `fill-compare.js`, `check-feeds.js`	kept with quant as they are: library verification, the catalogue, strategy checks and research scripts
the journal's start-up order, its minute timer and its environment (OPERATIONS)	desk-web's start-up; the scheduler's journal-sync job

8. The page shell

Legacy

A landing page on T1 (`home.html`, port 3000) linking every program; `screeners.html` (the suite), `manage.html` (Manage tools), `compare.html`; the journal on 3100 and qp on 8765. A shared app bar (`public/desk.js`, `desk.css`) names the programs; pages reading another port send the cookie (`desk.js` wraps `fetch`), with credentialed CORS for the same host. The journal could not take `desk.css` whole and served a sliced copy.

New design

- **One origin**, one app bar, one style sheet: `/` (landing), `/tools/…`, `/market`, `/journal`, `/quant`, `/health`, `/settings`.
- **Phone first**: every page readable on a phone; every label's ⓘ works by tap (CARD.md §5.6).
- No cross-port requests, so no CORS and no cookie wrapping.

---

9. Contracts

Owns

Table	Key
`users`, `sessions`	—
`jobs`, `job_runs`	job; job, start
`settings`, `setting_versions`	key, scope
`health_checks`	check, instant (with SOURCES.md)
`alerts_sent`	alert, instant
`backups`	night

Routes (desk-web)

Route	Does
`GET /login`, `POST /login`, `POST /logout`	the login
`GET /api/jobs`, `PATCH /api/jobs/:job`, `POST /api/jobs/:job/run`	the scheduler
`GET /api/jobs/:job/runs`	run history
`GET/PATCH /api/settings`	settings (checked)
`GET /api/health`, `POST /api/health/run`	health
`GET /api/backups`	backup history and verification

---

10. How the rebuild is proven

1. **Port scan.** From outside, only 443 answers.
2. **Missed-run test.** Disable the research worker’s job for a night: an alert arrives in the morning.
3. **Restore rehearsal.** The monthly rehearsal passes three months running before the old backup is retired.
4. **New-box test.** A fresh box restored with one command serves the same pages and numbers as the old one for a chosen past day.
5. **CI gate.** A deliberately failing test blocks a merge.
6. **Weekend test.** On a Monday before the open, and the morning after a holiday, no freshness check is amber.
7. **Overlap test.** A scan made to outlast its slot skips the next slot and records it; two scans never run at once.